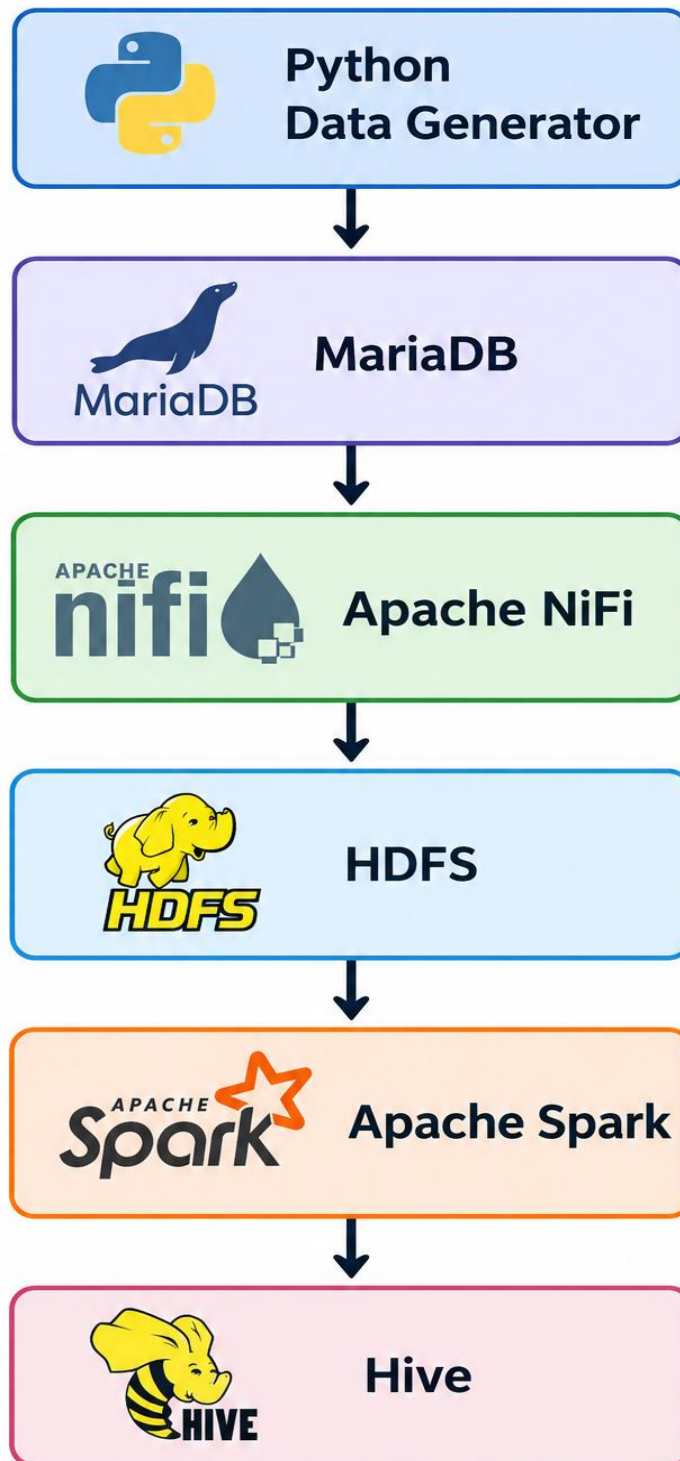


## End-to-End E-Commerce Data Engineering Pipeline



### ➤ Create the Database and Tables in MariaDB:

- Create the “ecommerce” database:

```
MariaDB [(none)]> CREATE DATABASE IF NOT EXISTS ecommerce;
Query OK, 1 row affected (0.01 sec)

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| ecommerce          |
| hive               |
| hue                |
| labs               |
| mysql              |
| performance_schema |
+-----+
7 rows in set (0.00 sec)
```

- Create “customers” table:

```
MariaDB [ecommerce]> CREATE TABLE customers (
->     customer_id BIGINT PRIMARY KEY,
->     first_name VARCHAR(100),
->     last_name VARCHAR(100),
->     email VARCHAR(255),
->     country VARCHAR(100),
->     city VARCHAR(100),
->     signup_date DATE
-> );
Query OK, 0 rows affected (0.00 sec)
```

- Create “products” table:

```
MariaDB [ecommerce]> CREATE TABLE products (  
->     product_id BIGINT PRIMARY KEY,  
->     product_name VARCHAR(255),  
->     category VARCHAR(100),  
->     price DECIMAL(10,2),  
->     stock_quantity INT  
-> );  
Query OK, 0 rows affected (0.01 sec)
```

- Create “orders” table:

```
MariaDB [ecommerce]> CREATE TABLE orders (  
->     order_id BIGINT PRIMARY KEY,  
->     customer_id BIGINT,  
->     order_date DATETIME,  
->     status VARCHAR(30),  
->     shipping_country VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.00 sec)
```

- Create “order\_items” table:

```
MariaDB [ecommerce]> CREATE TABLE order_items (  
->     order_item_id BIGINT PRIMARY KEY,  
->     order_id BIGINT,  
->     product_id BIGINT,  
->     quantity INT,  
->     unit_price DECIMAL(10,2)  
-> );  
Query OK, 0 rows affected (0.00 sec)
```

- Create “payments” table:

```
MariaDB [ecommerce]> CREATE TABLE payments (  
->     payment_id BIGINT PRIMARY KEY,  
->     order_id BIGINT,  
->     payment_method VARCHAR(50),  
->     payment_status VARCHAR(30),  
->     payment_date DATETIME,  
->     amount DECIMAL(10,2)  
-> );  
Query OK, 0 rows affected (0.00 sec)
```

- Show tables in the database:

```
MariaDB [ecommerce]> SHOW TABLES;  
+-----+  
| Tables_in_ecommerce |  
+-----+  
| customers            |  
| order_items          |  
| orders               |  
| payments             |  
| products             |  
+-----+  
5 rows in set (0.00 sec)
```

### ➤ Generate Data using Python and store it in MariaDB:

- Import needed libraries:

```
import mysql.connector
from faker import Faker
import random
from datetime import datetime, timedelta
```

- Configure the database and identify the number of records in each table:

```
# =====
# CONFIGURATION
# =====

DB_HOST = "localhost"
DB_PORT = 3306
DB_USER = "student"
DB_PASSWORD = "student"
DB_NAME = "ecommerce"

CUSTOMERS_COUNT = 100_000
PRODUCTS_COUNT = 20_000
ORDERS_COUNT = 500_000
ORDER_ITEMS_COUNT = 2_000_000
PAYMENTS_COUNT = 500_000

BATCH_SIZE = 5_000

fake = Faker()
```

- Connect to **MariaDB**:

```
# =====
# DATABASE CONNECTION
# =====

connection = mysql.connector.connect(
    host=DB_HOST,
    port=DB_PORT,
    user=DB_USER,
    password=DB_PASSWORD
)

cursor = connection.cursor()

connection.database = DB_NAME
```

# Hamza Mohamed Ali

---

- Generate the **customers** data and insert it into the “**customers**” table:

```
• # =====
• # 1. GENERATE CUSTOMERS
• # =====
•
• print("Generating customers...")
•
• countries = [
•     "Egypt",
•     "Saudi Arabia",
•     "UAE",
•     "Jordan",
•     "Kuwait",
•     "Qatar"
• ]
•
• customers = []
•
• for customer_id in range(1, CUSTOMERS_COUNT + 1):
•
•     first_name = fake.first_name()
•     last_name = fake.last_name()
•
•     customers.append((
•         customer_id,
•         first_name,
•         last_name,
•         f"{first_name.lower()}.{last_name.lower()}{customer_id}@example.com",
•         random.choice(countries),
•         fake.city(),
•         fake.date_between(
•             start_date="-5y",
•             end_date="today"
•         )
•     ))
•
• if len(customers) >= BATCH_SIZE:
•
•     cursor.executemany("""
•     INSERT INTO customers
•     VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
•     """, customers)
```

# Hamza Mohamed Ali

```
• connection.commit()
•
• customers = []
•
• print(f"Customers inserted: {customer_id:,}")
•
• # Insert remaining
• if customers:
•     cursor.executemany("""
•     INSERT INTO customers
•     VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
•     """, customers)
•
• connection.commit()
```

- Show “**customers**” table after inserting the generated data:

```
MariaDB [ecommerce]> SELECT * FROM customers LIMIT 5;
```

| customer_id | first_name | last_name | email                        | country      | city             | signup_date |
|-------------|------------|-----------|------------------------------|--------------|------------------|-------------|
| 1           | Julia      | Morris    | julia.morris1@example.com    | Jordan       | Burnsfort        | 2026-08-08  |
| 2           | Linda      | Rodriguez | linda.rodriguez2@example.com | Kuwait       | Lake Lesliemouth | 2023-03-01  |
| 3           | Heather    | Collins   | heather.collins3@example.com | Saudi Arabia | Davilaborough    | 2022-01-25  |
| 4           | Evelyn     | Weaver    | evelyn.weaver4@example.com   | Qatar        | New Richard      | 2023-01-25  |
| 5           | Kendra     | Lindsey   | kendra.lindsey5@example.com  | Qatar        | East Jamie       | 2024-08-30  |

```
5 rows in set (0.01 sec)
```

- Generate the **products** data and insert it into the “**products**” table:

```
• # =====
• # 2. GENERATE PRODUCTS
• # =====
•
• print("Generating products...")
•
• categories = [
•     "Electronics",
•     "Clothing",
•     "Shoes",
•     "Home",
•     "Beauty",
•     "Sports",
•     "Books",
```

```
•     "Gaming",
•     "Accessories",
•     "Furniture"
• ]
•
• products = []
•
• for product_id in range(1, PRODUCTS_COUNT + 1):
•
•     products.append((
•         product_id,
•         fake.catch_phrase(),
•         random.choice(categories),
•         round(random.uniform(5, 2000), 2),
•         random.randint(0, 1000)
•     ))
•
•     if len(products) >= BATCH_SIZE:
•
•         cursor.executemany("""
•             INSERT INTO products
•             VALUES (%s,%s,%s,%s,%s)
•             """, products)
•
•         connection.commit()
•
•         products = []
•
•         print(f"Products inserted: {product_id:,}")
•
• if products:
•
•     cursor.executemany("""
•         INSERT INTO products
•         VALUES (%s,%s,%s,%s,%s)
•         """, products)
•
•     connection.commit()
```

# Hamza Mohamed Ali

---

- Show “**products**” table after inserting the generated data:

```
MariaDB [ecommerce]> SELECT * FROM products LIMIT 5;
```

| product_id | product_name                        | category    | price   | stock_quantity |
|------------|-------------------------------------|-------------|---------|----------------|
| 1          | Front-line client-driven software   | Clothing    | 1187.44 | 968            |
| 2          | Configurable local hardware         | Accessories | 1032.38 | 908            |
| 3          | Sharable mobile time-frame          | Sports      | 1296.78 | 98             |
| 4          | Centralized bifurcated strategy     | Furniture   | 65.33   | 621            |
| 5          | User-friendly asymmetric task-force | Shoes       | 1145.05 | 26             |

```
5 rows in set (0.00 sec)
```

- Generate the **orders** data and insert it into the “**orders**” table:

```
• # =====
• # 3. GENERATE ORDERS
• # =====
•
• print("Generating orders...")
•
• statuses = [
•     "completed",
•     "completed",
•     "completed",
•     "pending",
•     "cancelled",
•     "shipped"
• ]
•
• orders = []
•
• start_date = datetime.now() - timedelta(days=730)
•
• for order_id in range(1, ORDERS_COUNT + 1):
•
•     customer_id = random.randint(1, CUSTOMERS_COUNT)
•
•     order_date = start_date + timedelta(
•         seconds=random.randint(
•             0,
•             730 * 24 * 60 * 60
•         )
•     )
•
• )
```

```
• orders.append((
•     order_id,
•     customer_id,
•     order_date,
•     random.choice(statuses),
•     random.choice(countries)
• ))
•
• if len(orders) >= BATCH_SIZE:
•
•     cursor.executemany("""
•     INSERT INTO orders
•     VALUES (%s,%s,%s,%s,%s)
•     """, orders)
•
•     connection.commit()
•
•     orders = []
•
•     print(f"Orders inserted: {order_id:,}")
•
• if orders:
•
•     cursor.executemany("""
•     INSERT INTO orders
•     VALUES (%s,%s,%s,%s,%s)
•     """, orders)
•
•     connection.commit()
```

- Show “**orders**” table after inserting the generated data:

```
MariaDB [ecommerce]> SELECT * FROM orders LIMIT 5;
+-----+-----+-----+-----+-----+
| order_id | customer_id | order_date          | status    | shipping_country |
+-----+-----+-----+-----+-----+
| 1 | 19599 | 2025-10-20 11:02:17 | completed | Egypt            |
| 2 | 19317 | 2025-08-12 18:26:54 | pending   | Kuwait           |
| 3 | 57859 | 2025-10-02 19:42:32 | cancelled | Saudi Arabia     |
| 4 | 64568 | 2024-10-01 22:55:31 | cancelled | Qatar            |
| 5 | 84679 | 2025-05-02 22:00:24 | completed | Egypt            |
+-----+-----+-----+-----+-----+
5 rows in set (0.02 sec)
```

- Generate the **order\_items** data and insert it into the “**order\_items**” table:

```
• # =====
• # 4. GENERATE ORDER ITEMS
• # =====
•
• print("Generating order items...")
•
• order_items = []
•
• for order_item_id in range(1, ORDER_ITEMS_COUNT + 1):
•
•     order_id = random.randint(1, ORDERS_COUNT)
•
•     product_id = random.randint(1, PRODUCTS_COUNT)
•
•     quantity = random.randint(1, 5)
•
•     unit_price = round(
•         random.uniform(5, 2000),
•         2
•     )
•
•     order_items.append((
•         order_item_id,
•         order_id,
•         product_id,
•         quantity,
•         unit_price
•     ))
•
•     if len(order_items) >= BATCH_SIZE:
•
•         cursor.executemany("""
•             INSERT INTO order_items
•             VALUES (%s,%s,%s,%s,%s)
•             """, order_items)
•
•         connection.commit()
•
•         order_items = []
•
•     print(
•         f"Order items inserted: "
•         f"{order_item_id:,}"
```

```
• )
•
• if order_items:
•
•     cursor.executemany("""
•     INSERT INTO order_items
•     VALUES (%s,%s,%s,%s,%s)
•     """, order_items)
•
•
•     connection.commit()
```

- Show “**order\_items**” table after inserting the generated data:

```
MariaDB [ecommerce]> SELECT * FROM order_items LIMIT 5;
+-----+-----+-----+-----+-----+
| order_item_id | order_id | product_id | quantity | unit_price |
+-----+-----+-----+-----+-----+
|          1   |    359084 |      16071 |         4 |      115.32 |
|          2   |    417538 |       9821 |         4 |     1578.51 |
|          3   |    298447 |      15262 |         2 |      634.09 |
|          4   |    308534 |       1107 |         1 |     1721.20 |
|          5   |    194989 |      15802 |         3 |     1258.99 |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

- Generate the **payments** data and insert it into the “**payments**” table:

```
• # =====
• # 5. GENERATE PAYMENTS
• # =====
•
• print("Generating payments...")
•
• payment_methods = [
•     "credit_card",
•     "debit_card",
•     "cash",
•     "paypal",
•     "bank_transfer"
• ]
•
• payment_statuses = [
```

```
•     "paid",
•     "paid",
•     "paid",
•     "failed",
•     "refunded"
• ]
•
• payments = []
•
• for payment_id in range(1, PAYMENTS_COUNT + 1):
•
•     order_id = payment_id
•
•     payment_date = start_date + timedelta(
•         seconds=random.randint(
•             0,
•             730 * 24 * 60 * 60
•         )
•     )
•
•     payments.append((
•         payment_id,
•         order_id,
•         random.choice(payment_methods),
•         random.choice(payment_statuses),
•         payment_date,
•         round(random.uniform(10, 5000), 2)
•     ))
•
•     if len(payments) >= BATCH_SIZE:
•
•         cursor.executemany("""
•             INSERT INTO payments
•             VALUES (%s,%s,%s,%s,%s,%s)
•             """, payments)
•
•         connection.commit()
•
•         payments = []
•
•         print(
•             f"Payments inserted: "
•             f"{payment_id:,"}
•         )
•
```

# Hamza Mohamed Ali

---

- `if payments:`
- 
- `cursor.executemany("""`
- `INSERT INTO payments`
- `VALUES (%s,%s,%s,%s,%s,%s)`
- `""", payments)`
- 
- `connection.commit()`

- Show “**payments**” table after inserting the generated data:

```
MariaDB [ecommerce]> SELECT * FROM payments LIMIT 5;
```

| payment_id | order_id | payment_method | payment_status | payment_date        | amount  |
|------------|----------|----------------|----------------|---------------------|---------|
| 1          | 1        | paypal         | refunded       | 2025-06-19 02:50:11 | 3943.68 |
| 2          | 2        | cash           | paid           | 2026-01-24 19:28:47 | 4429.51 |
| 3          | 3        | paypal         | refunded       | 2024-09-17 13:44:14 | 1587.00 |
| 4          | 4        | bank_transfer  | paid           | 2024-09-13 07:52:58 | 708.10  |
| 5          | 5        | bank_transfer  | paid           | 2025-07-08 03:44:43 | 3915.52 |

```
5 rows in set (0.00 sec)
```

- Close the connection with the database:

```
# =====  
# CLOSE CONNECTION  
# =====  
  
cursor.close()  
connection.close()  
  
print("=====")  
print("DATA GENERATION COMPLETED")  
print("=====")  
  
=====
```

DATA GENERATION COMPLETED

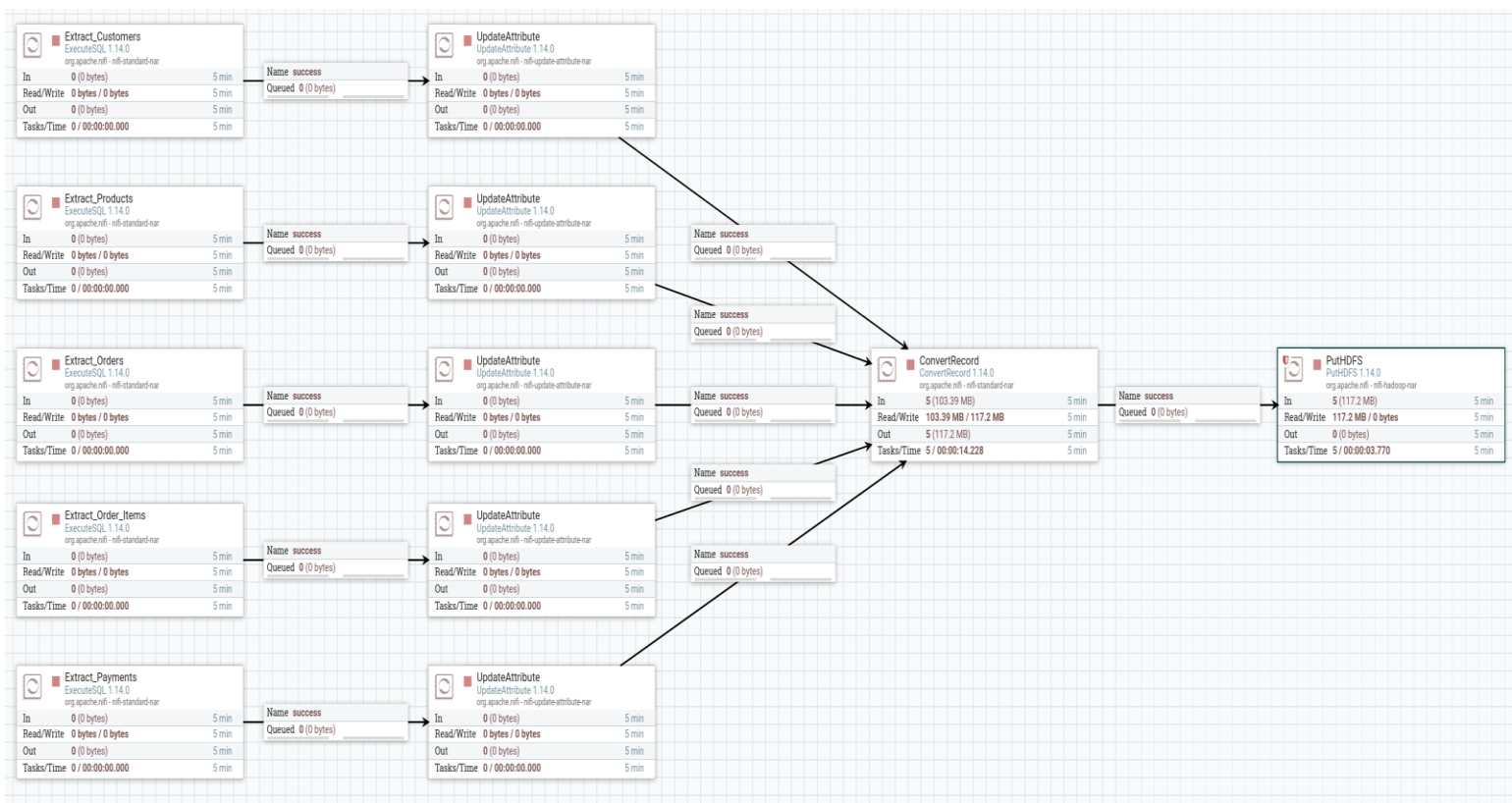
```
=====
```

## ➤ Extract Data using Apache NiFi and store it in a Staging Zone (HDFS):

- Build staging zone in **HDFS** for raw data:

```
[student@localhost ~]$ hdfs dfs -mkdir /user/student/ecommerce
[student@localhost ~]$ hdfs dfs -mkdir /user/student/ecommerce/staging_zone
[student@localhost ~]$ hdfs dfs -ls /user/student/ecommerce
Found 1 items
drwxr-xr-x  - student student    0 2026-09-07 23:59 /user/student/ecommerce/staging_zone
```

- NiFi Template:**



- Configure **MariaDB** connection:

Create a **Controller Service** for **MariaDB** connection:

| Property                     | Value                                   |  |
|------------------------------|-----------------------------------------|--|
| Database Connection URL      | jdbc:mariadb://localhost:3306/ecommerce |  |
| Database Driver Class Name   | org.mariadb.jdbc.Driver                 |  |
| Database Driver Location(s)  | No value set                            |  |
| Kerberos Credentials Service | No value set                            |  |
| Kerberos Principal           | No value set                            |  |
| Kerberos Password            | No value set                            |  |
| Database User                | student                                 |  |
| Password                     | Sensitive value set                     |  |

**Database Connection URL:** Provides the connection to database.

- **Processors:**

I use the same processors for each table to extract data from it and store it in its directory in **HDFS**.

- **For example:** processors for “**customers**” table:

## 1. **ExecuteSQL** processor named “**Extract\_customers**”:

To extract data from the table in **Avro** format.

|            |                                                                                      |       |
|------------|--------------------------------------------------------------------------------------|-------|
|            | <b>Extract_Customers</b><br>ExecuteSQL 1.14.0<br>org.apache.nifi - nifi-standard-nar |       |
| In         | 0 (0 bytes)                                                                          | 5 min |
| Read/Write | 0 bytes / 0 bytes                                                                    | 5 min |
| Out        | 0 (0 bytes)                                                                          | 5 min |
| Tasks/Time | 0 / 00:00:00.000                                                                     | 5 min |

Its configuration:

| Property                            | Value |                          |   |
|-------------------------------------|-------|--------------------------|---|
| Database Connection Pooling Service | ?     | MariaDB_Connection       | → |
| SQL Pre-Query                       | ?     | No value set             |   |
| SQL select query                    | ?     | SELECT * FROM customers; |   |
| SQL Post-Query                      | ?     | No value set             |   |

**Database Connection Pooling Service:** The **Controller Service** that is used to obtain connection to database.

**SQL select query:** A query to extract the data from “**customers**” table.

## 2. UpdateAttribute:

To rename the **FlowFile** after extraction by assigning it a descriptive and structured filename.



UpdateAttribute

UpdateAttribute 1.14.0

org.apache.nifi - nifi-update-attribute-nar

|            |                   |       |
|------------|-------------------|-------|
| In         | 0 (0 bytes)       | 5 min |
| Read/Write | 0 bytes / 0 bytes | 5 min |
| Out        | 0 (0 bytes)       | 5 min |
| Tasks/Time | 0 / 00:00:00.000  | 5 min |

Its configuration:

|          |   |               |
|----------|---|---------------|
| filename | ? | customers.csv |
|----------|---|---------------|

**filename:** Added property to change the **FlowFile** name.

## 3. ConvertRecord:

To convert the data format of the **FlowFile** from **Avro** to **CSV**.



ConvertRecord

ConvertRecord 1.14.0

org.apache.nifi - nifi-standard-nar

→ In

0 (0 bytes)

5 min

Read/Write

0 bytes / 0 bytes

5 min

Out

0 (0 bytes)

5 min

Tasks/Time

0 / 00:00:00.000

5 min

Its configuration:

| Property      | Value              |
|---------------|--------------------|
| Record Reader | AvroReader         |
| Record Writer | CSVRecordSetWriter |

**Record Reader:** Read the **FlowFile** in **Avro**.

**Record Writer:** Write the **FlowFile** in **CSV**.

## 4. PutHDFS:

To store the customers data in **HDFS**:



PutHDFS

PutHDFS 1.14.0

org.apache.nifi - nifi-hadoop-nar

|            |                   |       |
|------------|-------------------|-------|
| In         | 0 (0 bytes)       | 5 min |
| Read/Write | 0 bytes / 0 bytes | 5 min |
| Out        | 0 (0 bytes)       | 5 min |
| Tasks/Time | 0 / 00:00:00.000  | 5 min |

Its configuration:

|           |                                      |
|-----------|--------------------------------------|
| Directory | /user/student/ecommerce/staging_zone |
|-----------|--------------------------------------|

**Directory:** The directory to which file should be written.

## ➤ Verify that Files stored in HDFS:

```
[student@localhost ~]$ hdfs dfs -ls /user/student/ecommerce/staging_zone
Found 5 items
-rw-r--r--  1 root student    8220445 2026-09-08 00:04 /user/student/ecommerce/staging_zone/customers.csv
-rw-r--r--  1 root student   58238934 2026-09-08 00:04 /user/student/ecommerce/staging_zone/order_items.csv
-rw-r--r--  1 root student   25582342 2026-09-08 00:04 /user/student/ecommerce/staging_zone/orders.csv
-rw-r--r--  1 root student   29669506 2026-09-08 00:04 /user/student/ecommerce/staging_zone/payments.csv
-rw-r--r--  1 root student    1186477 2026-09-08 00:04 /user/student/ecommerce/staging_zone/products.csv
[student@localhost ~]$
```

## ➤ Transform Data using Apache Spark (PySpark):

- Read the files from their path and print their **schemas**:

```
customers = spark.read.format("csv") \
    .option("inferSchema", "true") \
    .option("header", "true") \
    .load("/user/student/ecommerce/staging_zone/customers.csv")

customers.printSchema()
customers.show(5)
```

```
root
|-- customer_id: integer (nullable = true)
|-- first_name: string (nullable = true)
|-- last_name: string (nullable = true)
|-- email: string (nullable = true)
|-- country: string (nullable = true)
|-- city: string (nullable = true)
|-- signup_date: string (nullable = true)
```

| customer_id | first_name | last_name | email                | country      | city             | signup_date |
|-------------|------------|-----------|----------------------|--------------|------------------|-------------|
| 1           | Julia      | Morris    | julia.morris1@exa... | Jordan       | Burnsfort        | 2026-08-08  |
| 2           | Linda      | Rodriguez | linda.rodriguez2@... | Kuwait       | Lake Lesliemouth | 2023-03-01  |
| 3           | Heather    | Collins   | heather.collins3@... | Saudi Arabia | Davilaborough    | 2022-01-25  |
| 4           | Evelyn     | Weaver    | evelyn.weaver4@ex... | Qatar        | New Richard      | 2023-01-25  |
| 5           | Kendra     | Lindsey   | kendra.lindsey5@e... | Qatar        | East Jamie       | 2024-08-30  |

only showing top 5 rows

```
products = spark.read.format("csv") \
    .option("inferSchema", "true") \
    .option("header", "true") \
    .load("/user/student/ecommerce/staging_zone/products.csv")

products.printSchema()
products.show(5)
```

```
root
|-- product_id: integer (nullable = true)
|-- product_name: string (nullable = true)
|-- category: string (nullable = true)
|-- price: double (nullable = true)
|-- stock_quantity: integer (nullable = true)
```

| product_id | product_name         | category    | price   | stock_quantity |
|------------|----------------------|-------------|---------|----------------|
| 1          | Front-line client... | Clothing    | 1187.44 | 968            |
| 2          | Configurable loca... | Accessories | 1032.38 | 908            |
| 3          | Sharable mobile t... | Sports      | 1296.78 | 98             |
| 4          | Centralized bifur... | Furniture   | 65.33   | 621            |
| 5          | User-friendly asy... | Shoes       | 1145.05 | 26             |

only showing top 5 rows

# Hamza Mohamed Ali

```
orders = spark.read.format("csv") \
    .option("inferSchema", "true") \
    .option("header", "true") \
    .load("/user/student/ecommerce/staging_zone/orders.csv")
```

```
orders.printSchema()
orders.show(5)
```

```
root
|-- order_id: integer (nullable = true)
|-- customer_id: integer (nullable = true)
|-- order_date: string (nullable = true)
|-- status: string (nullable = true)
|-- shipping_country: string (nullable = true)
```

| order_id | customer_id | order_date           | status    | shipping_country |
|----------|-------------|----------------------|-----------|------------------|
| 1        | 19599       | 2025-10-20 11:02:... | completed | Egypt            |
| 2        | 19317       | 2025-08-12 18:26:... | pending   | Kuwait           |
| 3        | 57859       | 2025-10-02 19:42:... | cancelled | Saudi Arabia     |
| 4        | 64568       | 2024-10-01 22:55:... | cancelled | Qatar            |
| 5        | 84679       | 2025-05-02 22:00:... | completed | Egypt            |

only showing top 5 rows

```
order_items = spark.read.format("csv") \
    .option("inferSchema", "true") \
    .option("header", "true") \
    .load("/user/student/ecommerce/staging_zone/order_items.csv")
```

```
order_items.printSchema()
order_items.show(5)
```

```
root
|-- order_item_id: integer (nullable = true)
|-- order_id: integer (nullable = true)
|-- product_id: integer (nullable = true)
|-- quantity: integer (nullable = true)
|-- unit_price: double (nullable = true)
```

| order_item_id | order_id | product_id | quantity | unit_price |
|---------------|----------|------------|----------|------------|
| 1             | 359084   | 16071      | 4        | 115.32     |
| 2             | 417538   | 9821       | 4        | 1578.51    |
| 3             | 298447   | 15262      | 2        | 634.09     |
| 4             | 308534   | 1107       | 1        | 1721.2     |
| 5             | 194989   | 15802      | 3        | 1258.99    |

only showing top 5 rows

- **Data Cleaning:**

- **Cleaning “customers” table:**

```
customers = customers.dropDuplicates()
customers = customers.withColumn("signup_date", to_date(col("signup_date"), "yyyy-MM-dd"))
customers.printSchema()

root
|-- customer_id: integer (nullable = true)
|-- first_name: string (nullable = true)
|-- last_name: string (nullable = true)
|-- email: string (nullable = true)
|-- country: string (nullable = true)
|-- city: string (nullable = true)
|-- signup_date: date (nullable = true)
```

- **Cleaning “products” table:**

```
products = products.dropDuplicates()
products = products.filter(
  (col("price").isNotNull()) & (col("price") > 0) &
  (col("stock_quantity").isNotNull()) & (col("stock_quantity") >= 0)
)
```

- **Cleaning “orders” table:**

```
orders = orders.dropDuplicates()
orders = orders.withColumn("order_date", to_timestamp(col("order_date"), "yyyy-MM-dd HH:mm:ss.S"))
orders = orders.withColumn("order_date", to_date(substring("order_date", 1, 10), "yyyy-MM-dd"))
orders.printSchema()

root
|-- order_id: integer (nullable = true)
|-- customer_id: integer (nullable = true)
|-- order_date: date (nullable = true)
|-- status: string (nullable = true)
|-- shipping_country: string (nullable = true)
```

- **Cleaning “order\_items” table:**

```
order_items = order_items.dropDuplicates()
order_items = order_items.filter(
  (col("quantity").isNotNull()) & (col("quantity") > 0) &
  (col("unit_price").isNotNull()) & (col("unit_price") >= 0)
)
```

- **Business Transformations:**

- Create a “**sales**” dataset:

```
sales = orders.join(order_items, "order_id", "inner") \
    .join(products, "product_id", "inner") \
    .join(customers, "customer_id", "inner")
```

- Add the **sales\_amount** column:

```
sales = sales.withColumn("sales_amount", col("quantity") * col("unit_price"))
```

```
sales.printSchema()
```

root

```
|-- customer_id: integer (nullable = true)
|-- product_id: integer (nullable = true)
|-- order_id: integer (nullable = true)
|-- order_date: date (nullable = true)
|-- status: string (nullable = true)
|-- shipping_country: string (nullable = true)
|-- order_item_id: integer (nullable = true)
|-- quantity: integer (nullable = true)
|-- unit_price: double (nullable = true)
|-- product_name: string (nullable = true)
|-- category: string (nullable = true)
|-- price: double (nullable = true)
|-- stock_quantity: integer (nullable = true)
|-- first_name: string (nullable = true)
|-- last_name: string (nullable = true)
|-- email: string (nullable = true)
|-- country: string (nullable = true)
|-- city: string (nullable = true)
|-- signup_date: date (nullable = true)
|-- sales_amount: double (nullable = true)
```

- **Total Sales:**

Around **6 billion**.

```
total_sales = sales.agg(  
    sum("sales_amount").alias("total_sales")  
)  
  
total_sales.show()
```

```
[Stage 144:=====
```

```
+-----+  
|      total_sales|  
+-----+  
|6.026882991339996E9|  
+-----+
```

- **Total Quantity Sold:**

```
total_quantity_sold = sales.agg(  
    sum("quantity").alias("total_quantity_sold")  
)  
  
total_quantity_sold.show()
```

```
[Stage 153:=====>
```

```
+-----+  
|total_quantity_sold|  
+-----+  
|          6001251|  
+-----+
```

- **Average Order Value:**

```
order_totals = sales.groupBy("order_id") \
    .agg(sum("sales_amount").alias("order_total"))

avg_order_value = order_totals.agg(
    avg("order_total").alias("avg_order_value")
)

avg_order_value.show()
```

[Stage 163:=====>

```
+-----+
| avg_order_value |
+-----+
| 12277.636404886702 |
+-----+
```

- **Number of Orders:**

```
number_of_orders = sales.select(
    countDistinct("order_id").alias("number_of_orders")
)

number_of_orders.show()
```

[Stage 173:=====>

```
+-----+
| number_of_orders |
+-----+
|           490883 |
+-----+
```

- **Number of customers:**

```
number_of_customers = sales.select(
    countDistinct("customer_id").alias("number_of_customers")
)

number_of_customers.show()
```

[Stage 183:=====>

```
+-----+
|number_of_customers|
+-----+
|                99232|
+-----+
```

- **Sales by Country:**

```
sales_by_country = sales.groupBy("country") \
    .agg(sum("sales_amount").alias("total_sales")) \
    .orderBy(desc("total_sales"))

sales_by_country.show()
```

[Stage 192:=====

```
+-----+-----+
|country|total_sales|
+-----+-----+
|Egypt|1.01824409189E9|
|UAE|1.0113604628000003E9|
|Saudi Arabia|1.0077514019000001E9|
|Jordan|1.0077170164500005E9|
|Kuwait|9.915593774700003E8|
|Qatar|9.902506408299999E8|
+-----+-----+
```

- Sales by Category:

```
sales_by_category = sales.groupBy("category") \
    .agg(sum("sales_amount").alias("total_sales")) \
    .orderBy(desc("total_sales"))
```

```
sales_by_category.show()
```

```
[Stage 201:=====>
```

| category    | total_sales         |
|-------------|---------------------|
| Clothing    | 6.2317556855E8      |
| Home        | 6.224572203699999E8 |
| Shoes       | 6.160828442200001E8 |
| Electronics | 6.131962863400002E8 |
| Books       | 6.003464328200002E8 |
| Accessories | 5.952419449999998E8 |
| Furniture   | 5.936714337299993E8 |
| Sports      | 5.902372777500001E8 |
| Beauty      | 5.881343664700001E8 |
| Gaming      | 5.843396160900003E8 |

- Sales by day:

```
sales_by_day = sales.groupBy("order_date") \
    .agg(sum("sales_amount").alias("total_sales")) \
    .orderBy("order_date")
```

```
sales_by_day.show()
```

```
[Stage 220:=====>
```

| order_date | total_sales         |
|------------|---------------------|
| 2024-09-07 | 3874759.9699999993  |
| 2024-09-08 | 8398860.1200000001  |
| 2024-09-09 | 8141191.18          |
| 2024-09-10 | 8253913.4900000001  |
| 2024-09-11 | 7913380.2100000002  |
| 2024-09-12 | 7851271.3899999994  |
| 2024-09-13 | 8476425.7499999996  |
| 2024-09-14 | 8682196.21          |
| 2024-09-15 | 8448867.6399999995  |
| 2024-09-16 | 8221108.7199999995  |
| 2024-09-17 | 8651925.86          |
| 2024-09-18 | 8255038.52999999975 |
| 2024-09-19 | 8087594.89999999985 |
| 2024-09-20 | 9039002.7899999997  |
| 2024-09-21 | 8469588.0799999998  |
| 2024-09-22 | 8246190.0699999999  |
| 2024-09-23 | 8159726.35          |
| 2024-09-24 | 8993464.88          |
| 2024-09-25 | 7662265.1700000001  |
| 2024-09-26 | 7817639.2099999998  |

```
only showing top 20 rows
```

- **Sales by Month:**

```
sales_by_month = sales.withColumn("month", month("order_date")) \
    .withColumn("year", year("order_date")) \
    .groupBy("month", "year") \
    .agg(sum("sales_amount").alias("total_sales")) \
    .orderBy("year", "month")
```

```
sales_by_month.show(10)
```

[Stage 282:=====> (1

```
+---+---+-----+
|month|year|      total_sales|
+---+---+-----+
|  9|2024|1.9331199270000017E8|
| 10|2024|2.5422639881000003E8|
| 11|2024|2.4948408238999984E8|
| 12|2024|2.5577841497999996E8|
|  1|2025|2.5591715853999996E8|
|  2|2025|2.2934754572000015E8|
|  3|2025| 2.560887445300001E8|
|  4|2025|2.4876624536000004E8|
|  5|2025|2.5521503787000018E8|
|  6|2025|2.5071554162000012E8|
+---+---+-----+
```

only showing top 10 rows

- **Sales by Year:**

```
sales_by_year = sales.withColumn("year", year("order_date")) \
    .groupBy("year") \
    .agg(sum("sales_amount").alias("total_sales")) \
    .orderBy("year")
```

```
sales_by_year.show()
```

[Stage 300:=====>

```
+---+-----+
|year|      total_sales|
+---+-----+
|2024| 9.528008888800011E8|
|2025|3.0177951298799996E9|
|2026|2.0562869725800004E9|
+---+-----+
```

## ➤ Create Analytics Tables:

- **Fact Sales:**

```
fact_sales = sales.select(
    "order_id", "customer_id", "product_id", "order_date", "quantity",
    "unit_price", col("sales_amount").alias("total_amount"),
    "category", "country", "status"
)
```

```
fact_sales.show(5)
```

[Stage 345:=====>(197 + 3) / 200]

| order_id | customer_id | product_id | order_date | quantity | unit_price | total_amount       | category    | country | status    |
|----------|-------------|------------|------------|----------|------------|--------------------|-------------|---------|-----------|
| 148      | 10063       | 5760       | 2025-06-13 | 5        | 1699.61    | 8498.05            | Electronics | Egypt   | completed |
| 148      | 10063       | 15560      | 2025-06-13 | 5        | 1302.66    | 6513.3             | Electronics | Egypt   | completed |
| 148      | 10063       | 19309      | 2025-06-13 | 3        | 1273.4     | 3820.2000000000003 | Clothing    | Egypt   | completed |
| 148      | 10063       | 4398       | 2025-06-13 | 4        | 1909.62    | 7638.48            | Accessories | Egypt   | completed |
| 148      | 10063       | 15315      | 2025-06-13 | 3        | 1137.62    | 3412.8599999999997 | Furniture   | Egypt   | completed |

only showing top 5 rows

- **Daily Sales:**

```
daily_order_totals = sales.withColumn("year", year("order_date")) \
    .withColumn("month", month("order_date")) \
    .withColumn("day", dayofmonth("order_date")) \
    .groupBy("year", "month", "day", "order_id") \
    .agg(
        sum("quantity").alias("order_quantity"),
        sum("sales_amount").alias("order_total")
    )
```

```
daily_sales = daily_order_totals.groupBy("year", "month", "day") \
    .agg(
        countDistinct("order_id").alias("total_orders"),
        sum("order_quantity").alias("total_quantity"),
        sum("order_total").alias("total_sales"),
        avg("order_total").alias("avg_order_value")
    ) \
    .orderBy("year", "month", "day")
```

```
daily_sales.show(5)
```

[Stage 393:=====> (188 + 7) / 200]

| year | month | day | total_orders | total_quantity | total_sales       | avg_order_value    |
|------|-------|-----|--------------|----------------|-------------------|--------------------|
| 2024 | 9     | 7   | 314          | 3813           | 3874759.970000002 | 12339.999904458606 |
| 2024 | 9     | 8   | 678          | 8286           | 8398860.120000003 | 12387.699292035402 |
| 2024 | 9     | 9   | 677          | 8150           | 8141191.180000003 | 12025.393175775485 |
| 2024 | 9     | 10  | 662          | 8204           | 8253913.49        | 12468.147265861027 |
| 2024 | 9     | 11  | 663          | 7835           | 7913380.209999998 | 11935.716757164402 |

only showing top 5 rows

- **Customer Sales:**

```
customer_sales = sales.withColumn("customer_name", concat_ws(" ", col("first_name"), col("last_name"))) \
    .groupBy("customer_id", "customer_name", "country") \
    .agg(
        countDistinct("order_id").alias("total_orders"),
        sum("quantity").alias("total_quantity"),
        sum("sales_amount").alias("total_spending")
    ) \
    .orderBy(col("total_spending").desc())

customer_sales.show(5)
```

| customer_id | customer_name    | country      | total_orders | total_quantity | total_spending     |
|-------------|------------------|--------------|--------------|----------------|--------------------|
| 60985       | Ryan Hall        | Jordan       | 13           | 210            | 242648.48          |
| 70998       | Michael Wilson   | Jordan       | 13           | 217            | 241643.88          |
| 62026       | Chase Brown      | Saudi Arabia | 14           | 207            | 241031.72          |
| 47789       | Emily Thomas     | Kuwait       | 15           | 229            | 239637.10999999996 |
| 81473       | Jessica Crawford | Qatar        | 13           | 200            | 229712.86000000002 |

only showing top 5 rows

- **Product Sales:**

```
product_sales = sales.groupBy("product_id", "product_name", "category") \
    .agg(
        sum("quantity").alias("total_quantity"),
        sum("sales_amount").alias("total_sales")
    ) \
    .orderBy(col("total_sales").desc())

product_sales.show(5)
```

[Stage 421:=====>(199 + 1) / 200]

| product_id | product_name         | category    | total_quantity | total_sales        |
|------------|----------------------|-------------|----------------|--------------------|
| 11771      | Reduced web-enabl... | Shoes       | 407            | 477799.45999999999 |
| 12608      | Integrated homoge... | Clothing    | 421            | 473207.62999999998 |
| 1912       | Balanced 5thgener... | Accessories | 431            | 466242.88          |
| 232        | Up-sized human-re... | Home        | 426            | 463075.84          |
| 17293      | Fully-configurabl... | Beauty      | 416            | 446674.06000000002 |

only showing top 5 rows

## ➤ Load Data into Hive:

- Create “ecommerce\_dw” database in **Hive**:

```
spark.sql("CREATE DATABASE IF NOT EXISTS ecommerce_dw")
```

```
spark.sql("SHOW DATABASES").show()
```

```
+-----+
| namespace |
+-----+
|      default      |
|      ecommerce    |
| ecommerce_dw      |
|      retail_db    |
+-----+
```

- Load the **Analytics Tables** in **Hive**:

```
fact_sales.write \
  .mode("overwrite") \
  .format("parquet") \
  .option("compression", "snappy") \
  .saveAsTable("ecommerce_dw.fact_sales")
```

```
2026-09-09 11:36:12,018 WARN session.SessionState: METASTORE_FILTER_HOOK will be ignored, since hive.security.authorization.manager is set to instance of HiveAuthorizerFactory.
2026-09-09 11:36:12,193 WARN conf.HiveConf: HiveConf of name hive.internal.ss.authz.settings.applied.marker does not exist
2026-09-09 11:36:12,193 WARN conf.HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist
2026-09-09 11:36:12,194 WARN conf.HiveConf: HiveConf of name hive.stats.retries.wait does not exist
```

```
daily_sales.write \
  .mode("overwrite") \
  .format("parquet") \
  .option("compression", "snappy") \
  .saveAsTable("ecommerce_dw.daily_sales")
```

```
customer_sales.write \
  .mode("overwrite") \
  .format("parquet") \
  .option("compression", "snappy") \
  .saveAsTable("ecommerce_dw.customer_sales")
```

```
product_sales.write \
  .mode("overwrite") \
  .format("parquet") \
  .option("compression", "snappy") \
  .saveAsTable("ecommerce_dw.product_sales")
```

➤ Data Validation Queries:

- Fact Sales:

```
1 SELECT *
2 FROM ecommerce_dw.fact_sales
3 LIMIT 10;
```

Output:

|    | fact_sales.order_id | fact_sales.customer_id | fact_sales.product_id | fact_sales.order_date | fact_sales.quantity |
|----|---------------------|------------------------|-----------------------|-----------------------|---------------------|
| 1  | 148                 | 10063                  | 5760                  | 2025-06-13            | 5                   |
| 2  | 148                 | 10063                  | 15560                 | 2025-06-13            | 5                   |
| 3  | 148                 | 10063                  | 19309                 | 2025-06-13            | 3                   |
| 4  | 148                 | 10063                  | 4398                  | 2025-06-13            | 4                   |
| 5  | 148                 | 10063                  | 15315                 | 2025-06-13            | 3                   |
| 6  | 148                 | 10063                  | 526                   | 2025-06-13            | 4                   |
| 7  | 463                 | 122                    | 16415                 | 2025-04-20            | 1                   |
| 8  | 463                 | 122                    | 1457                  | 2025-04-20            | 2                   |
| 9  | 471                 | 11344                  | 17285                 | 2024-12-03            | 3                   |
| 10 | 471                 | 11344                  | 16339                 | 2024-12-03            | 1                   |

| fact_sales.unit_price | fact_sales.total_amount | fact_sales.category | fact_sales.country | fact_sales.status |
|-----------------------|-------------------------|---------------------|--------------------|-------------------|
| 1699.61               | 8498.05                 | Electronics         | Egypt              | completed         |
| 1302.66               | 6513.3                  | Electronics         | Egypt              | completed         |
| 1273.4                | 3820.2000000000003      | Clothing            | Egypt              | completed         |
| 1909.62               | 7638.48                 | Accessories         | Egypt              | completed         |
| 1137.62               | 3412.8599999999997      | Furniture           | Egypt              | completed         |
| 1789.27               | 7157.08                 | Shoes               | Egypt              | completed         |
| 1165.37               | 1165.37                 | Beauty              | Qatar              | completed         |
| 1192.67               | 2385.34                 | Books               | Qatar              | completed         |
| 1060.65               | 3181.9500000000003      | Beauty              | Egypt              | shipped           |
| 608.67                | 608.67                  | Electronics         | Egypt              | shipped           |

• Daily Sales:

```
5 SELECT *
6 FROM ecommerce_dw.daily_sales
7 LIMIT 10;
```

Output:

|    | daily_sales.year | daily_sales.month | daily_sales.day | daily_sales.total_orders | daily_sales.total_quantity |
|----|------------------|-------------------|-----------------|--------------------------|----------------------------|
| 1  | 2024             | 9                 | 7               | 314                      | 3813                       |
| 2  | 2024             | 9                 | 8               | 678                      | 8286                       |
| 3  | 2024             | 9                 | 9               | 677                      | 8150                       |
| 4  | 2024             | 9                 | 10              | 662                      | 8204                       |
| 5  | 2024             | 9                 | 11              | 663                      | 7835                       |
| 6  | 2024             | 9                 | 12              | 648                      | 7805                       |
| 7  | 2024             | 9                 | 13              | 683                      | 8397                       |
| 8  | 2024             | 9                 | 14              | 717                      | 8819                       |
| 9  | 2024             | 9                 | 15              | 681                      | 8188                       |
| 10 | 2024             | 9                 | 16              | 649                      | 8177                       |

| daily_sales.total_sales | daily_sales.avg_order_value |
|-------------------------|-----------------------------|
| 3874759.9700000002      | 12339.999904458606          |
| 8398860.1200000003      | 12387.699292035402          |
| 8141191.1800000003      | 12025.393175775485          |
| 8253913.49              | 12468.147265861027          |
| 7913380.2099999998      | 11935.716757164402          |
| 7851271.39000000015     | 12116.159552469138          |
| 8476425.7500000004      | 12410.579428989757          |
| 8682196.2099999997      | 12109.060264993022          |
| 8448867.6399999999      | 12406.560411160057          |
| 8221108.7200000002      | 12667.347796610173          |

- **Customer Sales:**

```
9 SELECT *
10 FROM ecommerce_dw.customer_sales
11 LIMIT 10;
```

Output:

|    | customer_sales.customer_id | customer_sales.customer_name | customer_sales.country | customer_sales.total_orders |
|----|----------------------------|------------------------------|------------------------|-----------------------------|
| 1  | 60985                      | Ryan Hall                    | Jordan                 | 13                          |
| 2  | 70998                      | Michael Wilson               | Jordan                 | 13                          |
| 3  | 62026                      | Chase Brown                  | Saudi Arabia           | 14                          |
| 4  | 47789                      | Emily Thomas                 | Kuwait                 | 15                          |
| 5  | 81473                      | Jessica Crawford             | Qatar                  | 13                          |
| 6  | 65548                      | Autumn Newton                | Saudi Arabia           | 11                          |
| 7  | 56683                      | Steven McLaughlin            | Qatar                  | 12                          |
| 8  | 9912                       | Stephanie Figueroa           | Egypt                  | 14                          |
| 9  | 31406                      | Jonathan Madden              | UAE                    | 14                          |
| 10 | 99518                      | Douglas McCarthy             | Egypt                  | 11                          |

| customer_sales.total_quantity | customer_sales.total_spending |
|-------------------------------|-------------------------------|
| 210                           | 242648.48                     |
| 217                           | 241643.88                     |
| 207                           | 241031.72                     |
| 229                           | 239637.10999999996            |
| 200                           | 229712.86000000002            |
| 190                           | 228610.12999999995            |
| 194                           | 219815.02999999997            |
| 201                           | 219369.87                     |
| 205                           | 216602.22999999998            |
| 195                           | 215328.82                     |

- **Product Sales:**

```
13 SELECT *
14 FROM ecommerce_dw.product_sales
15 LIMIT 10;
```

Output:

|    | product_sales.product_id | product_sales.product_name              | product_sales.category |
|----|--------------------------|-----------------------------------------|------------------------|
| 1  | 11771                    | Reduced web-enabled knowledge user      | Shoes                  |
| 2  | 12608                    | Integrated homogeneous throughput       | Clothing               |
| 3  | 1912                     | Balanced 5thgeneration application      | Accessories            |
| 4  | 232                      | Up-sized human-resource frame           | Home                   |
| 5  | 17293                    | Fully-configurable intermediate concept | Beauty                 |
| 6  | 16608                    | Monitored systematic extranet           | Accessories            |
| 7  | 925                      | Visionary local middleware              | Clothing               |
| 8  | 17522                    | Pre-emptive homogeneous approach        | Beauty                 |
| 9  | 19309                    | Exclusive neutral utilization           | Clothing               |
| 10 | 10290                    | Down-sized attitude-oriented capability | Books                  |

| product_sales.total_quantity | product_sales.total_sales |
|------------------------------|---------------------------|
| 407                          | 477799.4599999999         |
| 421                          | 473207.62999999998        |
| 431                          | 466242.88                 |
| 426                          | 463075.84                 |
| 416                          | 446674.06000000002        |
| 407                          | 443845.079999999984       |
| 431                          | 441892.31                 |
| 398                          | 440973.26                 |
| 393                          | 438457.30999999999        |
| 385                          | 435585.31000000001        |